Waiter System (WaS1) Workshop 4 [25]

THREE-Tier Application

Objectives

The main objectives of this **pilot system** are to enhance your understanding of 3 tier applications and OO principles and to enable you to

- **Implement good OO programming principles**
- **Use inheritance and composition.**
- **Use collections**

1. Include a class *EmployeeDB* in your Database Layer folder to allow data manipulation.

1.1 Ensure class *EmployeeDB* inherits from the *DB* class (read your notes on inheritance!)

1.2 The *EmployeeDB* has data members to define the specific table in the database and the SQL statements associated with that table. Declare the data members for all of the role types. For example, *table1* will represent the *HeadWaiter* table, and the SQL statement is a SQL command to show all the data/fields from the HeadWaiter table:

```
private string table1 = "HeadWaiter";
private string sqlLocal1 = "SELECT * FROM HeadWaiter";
```

1.3 Declare a collection *employees* of Employees: private Collection<Employee> employees; (this works similar to a List)

1.4 Add a property (get) method *AllEmployees* that will return a collection of employees

1.5 Add a default constructor to the class; and within the constructor:

1.5.1 Create an instance of the employee collection: employees = new Collection<Employee>():

1.5.2 Fill the Dataset (for example for Headwaiter: using *sqlLocal1* as the sql statement and *table1* as the table*)* – *simply c*all the FillDataSet method that you defined previously;

1.6 Start a new region titled *Utility Methods*.

1.6.1 Add a method *GetDataSet* of data type DataSet, to return the current data set *(dsMain* in the base class*)* .

1.6.2 Add a method *Add2Collection* to build a collection of all the employees (Headwaiters, Waiters and Runners). This collection will be filled with data (object by object) from each row (record) in one of the tables: HeadWaiter, Waiter and Runner. The method therefore has one string parameter *table* to denote the specific table. Within this method:

1.6.2.1 Declare a reference to a *myRow* object (Data type DataRow) and assign it null

1.6.2.2 Declare reference to an Employee object: Employee *anEmp*;

1.6.2.3 Similarly, declare reference to each of the derived classes of Role as follows:
HeadWaiter headW; Waiter waiter; Runner runner;

1.6.2.4 Declare *roleValue* and initialise it to No role.

1.6.2.5 Use a switch statement to switch between the different tables. In case the table is a HeadWaiter then roleValue = Role.RoleType.Headwaiter. Do the same for all tables.

1.6.2.6 Use a **foreach** statement to read from the table as follows:
foreach (DataRow myRow_loopVariable in dsMain.Tables[table].Rows)

1.6.2.7 Within the foreach statement:

a) Assign myRow_loopVariable to myRow

b) Use an if statement to check the RowState of a myRow. See also DataRowState. If the row's state is not *DataRowState.Deleted* then

i. Instantiate a new Employee object. Note the Employee object required the role value

ii. Obtain each employee attribute from the specific field in the row in the table as follows: anEmp.ID = Convert.ToString(myRow["ID"]).TrimEnd();

```
//Do the same for all other attributes
```

With regards to the Role attribute, depending on Role, a switch statement needs to be used because different attributes come into play. An example is given for HeadWaiter. You can do the others.

```
switch (anEmp.role.RoleValue)
{
   case Role.RoleType.Headwaiter:
      headW = (HeadWaiter)anEmp.role;
      headW.Salary = Convert.ToDecimal(myRow["Salary"]);
      break;
```

iii. Once out of the switch statement, add the employee object to the collection of employees

1.6.3 Add a method FillRow to populate the row. The method has two parameters: *aRow* of type DataRow, and an employee object *anEmp*. Within this method:

1.6.3.1 Declare 3 variables to capture each of the Employee types; for example,

HeadWaiter headwaiter;

1.6.3.2 Assign the attributes of the employee object to the data table field. An example is given below:

```
aRow["ID"] = anEmp.ID;  //NOTE square brackets to indicate index of collections of fields in row.

aRow["EmpID"] = anEmp.EmpID;
```

When adding the Role field, you will need to check the role of each employee, because the salary changes per role. For each role add the specific data variables.An example is given below.

```
switch (anEmp.role.RoleValue)
      {
         case Role.RoleType.Headwaiter:
            headwaiter = (HeadWaiter)anEmp.role;
            aRow["Salary"] = headwaiter.Salary;
            break;
```

1.7 In the Constructor of the class, just after calling the FillDataSet() method:

1.7.1 Call the Add2Collection method associated with Table 1 as follows: Add2Collection(table1);

1.7.2 Call the *FillDataSet()* method with sqlLocal2 and Table *table2* as parameters. And add this to the collection (by calling the Add2Collection method associated with Table 2)

1.7.3 Finally, call the *FillDataSet()* method with sqlLocal3 and Table *table3* as parameters. And add this to the collection (by calling the Add2Collection method associated with Table 3)

1.8 Start a new region titled *Database Operations CRUD* to add the object's values to the database. Create a method *DataSetChange* which has one parameter – the Employee object called *anEmp*. In this case, dataset change refers to adding to a database table. Within this method:

1.8.1 Declare a variable *aRow* of type DataRow and initialise it to null.

1.8.2 Declare a variable *dataTable* and initialise it to *table1*

1.8.3 Use a switch statement to detect which table is being referred to (The switch statement will need the RoleValue of the employee object to alternative between the roles). In case the Role is headwaiter, then the dataTable should be assigned table1; etc

a) After determining the role of the employee, create a new row in the dataset as follows:

```
aRow =dsMain.Tables[dataTable].NewRow();
```

b) Call the *FillRow* method. This method needs a row and an employee object

c) Finally, add the new row to the dataset as follows: dsMain.Tables[dataTable].Rows.Add(aRow);

Then you can go to the next case of the switch statement. The code for the other cases is given below:

```
        case Role.RoleType.Waiter:
          dataTable = table2;
          break;
      case Role.RoleType.Runner:
          dataTable = table3;
             break;
```

}

1.9 Start a new region titled Build Parameters, Create Commands & Update database. Three methods will be defined here. We will use the SqlClient Parameters. See the section on Working with SqlClient Parameters; and also run an INSERT statement with parameters

1.9.1 Define a method *Build_INSERT_Parameters*. This method receives an Employee object *anEmp*. Within this method:

1.9.1.1 create parameters to communicate with SQL INSERT. An example is given below

```
SqlParameter param = default(SqlParameter);
param = new SqlParameter("@ID", SqlDbType.NVarChar, 15, "ID");
daMain.InsertCommand.Parameters.Add(param);
param = new SqlParameter("@EMPID", SqlDbType.NVarChar, 10, "ID");
daMain.InsertCommand.Parameters.Add(param);
```

Now do for all the other fields. The following figure is useful to explain some of the keywords used

The following table shows the DataRowVersion enumeration values available for use with `SourceVersion`.

DataRowVersion Enumeration	Description
`Current`	The parameter uses the current value of the column. This is the default.
`Default`	The parameter uses the `DefaultValue` of the column.
`Original`	The parameter uses the original value of the column.
`Proposed`	The parameter uses a proposed value.

1.9.1.2 After creating the parameters for Role, you will need a switch statement to determine the role as follows:

```
switch (anEmp.role.RoleValue)
        {
           case Role.RoleType.Headwaiter:
              param = new SqlParameter("@Salary", SqlDbType.Money, 8, "Salary");
              daMain.InsertCommand.Parameters.Add(param);
              break;
```

Now do for the other roles. For more on SQLDBTypes visit https://msdn.microsoft.com/en-za/library/ms179882.aspx

1.9.2 Define a method *Create_INSERT_Command* which receives an Employee object *anEmp.* In this method we will create the commands that must be used to insert values into the tables. Within this method:

1.9.2.1 You will need a switch statement to determine the role of the employee that needs to be added to the table…. An example is given below, you can do for the others

```
switch (anEmp.role.RoleValue)
        {
           case Role.RoleType.Headwaiter:
              daMain.InsertCommand = new SqlCommand("INSERT into HeadWaiter (ID, EMPID, Name,
                                                                    Phone,Role,Salary)
           VALUES (@ID, @EmpID, @Name, @Phone, @Role, @Salary)", cnMain);
              break;
```

1.9.2.2 After the switch statement, call the *Build_INSERT_Parameters* method, giving it an employee object.

1.9.3 Define a (public) boolean method *UpdateDataSource* which receives an Employee object *anEmp*. Within this method:

1.9.3.1 Define a boolean variable success and assign it the value *true*

1.9.3.2 Call the Create_INSERT_Command, giving it an employee object.

1.9.3.3 Depending on the role of the employee (therefore use a switch or any equivalent), call the UpdateDataSource method (which is in the DB class) and assign it to the variable success.

1.9.3.4 The method then returns success value to the calling procedure.

2. Section B: Application Layer

2.1 Add a new class *EmployeeController* to the Business Layer.

2.2 Declare a reference to the *EmployeeDB* class

2.3 Declare a collection of *employees*

2.4 In the constructor, instantiate the *EmployeeDB* object to communicate with the database. So first instantiate the *EmployeeD*B; and then assign the database employees to the collection.

2.5 Start a new region titled Database Communication.

2.5.1 Define a method in here called *DataMaintenance* which has an employee object *anEmp*. Within this method:

2.5.1.1 add a new employee object to the dataset in memory by calling the *DataSetChange* method

2.5.1.2 and add to the employees collection.

2.5.2 Define a Boolean method *FinalizeChanges* which has an employee object *employee.* This method will commit the changes to the database. Within this method, call the *EmployeeDB* method that will commit the changes to the database (i.e return the *EmployeeDB* method that will commit the changes to the database)